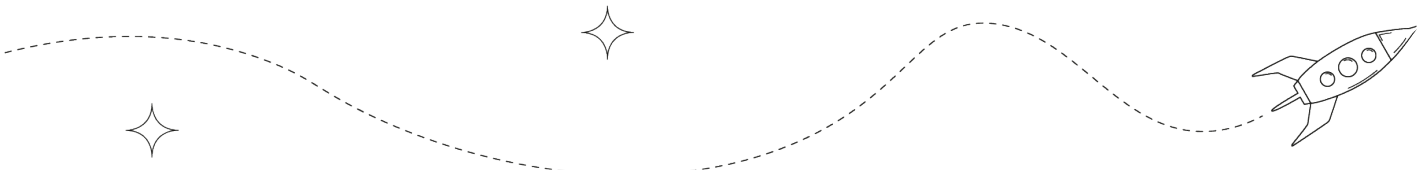
MISE EN SERVICE DU WMS STRUCTA SUR SERVEUR

OBJECTIFS

Décrire pas à pas l'installation du WMS Structa sur une machine serveur du réseau de l'entrepôt : de la préparation de la machine jusqu'à l'ouverture du service aux opérateurs, en passant par la base de données, la reprise des données existantes et la sauvegarde.
Deux exigences commandent l'ensemble : aucune donnée ne doit être perdue à la reprise, et le service doit repartir seul après une coupure de courant, sans que personne ait à ouvrir une session sur le serveur.

DESTINATAIRES

Service informatique, responsable logistique, et toute personne appelée à installer, redémarrer ou dépanner le WMS Structa.



N° Ed.	*Date*	*Description des modifications*	*Rédigé par*	*Vérifié par*	*Approuvé par*

1	*25/08/26*	*Création*	*VH*	*—*	*—*

Table des matières

1. AVANT DE COMMENCER....2
2. PRÉPARER LA MACHINE SERVEUR....3
3. CRÉER LA BASE POSTGRESQL....3
4. REPRENDRE LES DONNÉES EXISTANTES....3
5. INSTALLER LE SERVICE....3
6. VÉRIFIER AVANT D'OUVRIR AUX OPÉRATEURS....3
7. SAUVEGARDER....3
8. SUPERVISER ET DÉPANNER....3

1. AVANT DE COMMENCER

Cette procédure suppose que le WMS fonctionne déjà sur un poste, avec ses données. Elle décrit le passage de ce poste à une machine dédiée, allumée en permanence, que les téléphones de l'entrepôt interrogeront par le réseau.

Comptez une demi-journée pour la première mise en service. Réunissez d'abord :

- le dossier du projet complet, copié depuis le poste actuel ;
- le fichier db.sqlite3 du poste actuel, s'il contient des données à reprendre ;
- un accès administrateur sur la machine serveur ;
- l'adresse IP que prendra le serveur, fixée à l'avance.

> **Ne copiez jamais le fichier .env du poste vers le serveur. Il contient la clé secrète qui signe les sessions : partagée entre deux machines, une session forgée sur l'une fonctionne sur l'autre. Le serveur génère la sienne.**

2. PRÉPARER LA MACHINE SERVEUR

Un PC dédié suffit : 4 Go de mémoire, un disque SSD, et surtout allumé en permanence. Ce dernier point n'est pas un détail — c'est ce qui distingue un serveur d'un poste de travail.

Installer, dans cet ordre :

- Python 3.12 ou plus récent, en cochant « Add Python to PATH » ;
- PostgreSQL 16, depuis postgresql.org/download/windows. Noter le mot de passe du compte postgres demandé à l'installation, et cocher « Command Line Tools » : c'est cette case qui fournit psql et pg_dump ;
- le dossier du projet, par exemple dans C:\WMS\wms-structa.

Fixer enfin une adresse IP fixe à la machine, ou une réservation DHCP côté box. Les téléphones se connecteront à cette adresse : elle ne doit pas changer.

3. CRÉER LA BASE POSTGRESQL

Le plus simple est le double-clic sur Basculer-PostgreSQL.bat, à la racine du projet. Il crée le rôle applicatif et la base d'exploitation, bascule la configuration, applique

le schéma et reprend les données — les sections 3 et 4 de cette procédure en un seul geste.

Il ne détruit jamais rien : si une base wms existe déjà et contient des tables, il s'arrête et vous laisse décider. Il vérifie aussi que le service PostgreSQL répond avant de demander le moindre mot de passe.

Pour procéder à la main, ouvrir deploiement\postgresql\creer-base.sql, y remplacer le mot de passe d'exemple, puis :

```
psql -U postgres -f deploiement\postgresql\creer-base.sql
```

Le script crée un rôle wms et une base wms qui lui appartient. L'application ne se connecte jamais en postgres : un défaut applicatif deviendrait sinon un pouvoir d'administration sur toutes les bases de la machine.

Renseigner ensuite le fichier .env :

```
DJANGO_DB_ENGINE=django.db.backends.postgresql
DJANGO_DB_NAME=wms
DJANGO_DB_USER=wms
DJANGO_DB_PASSWORD=<le mot de passe choisi>
DJANGO_DB_HOST=127.0.0.1
DJANGO_DB_PORT=5432
```

Vérifier l'accès avant d'aller plus loin :

```
psql -U wms -d wms -h 127.0.0.1 -f deploiement\postgresql\verifier-acces.sql
```

4. REPRENDRE LES DONNÉES EXISTANTES

C'est l'étape à risque. Pas parce qu'elle peut échouer : parce qu'elle peut réussir à moitié. Une base d'apparence normale, mais amputée d'une table que personne ne remarque avant le jour où une cabine manque à l'appel.

L'outil de reprise s'en prémunit par un manifeste de comptages : l'export compte les lignes de chaque table, l'import recompte après chargement, et annule tout au moindre écart.

```
rem 1. AVANT de basculer .env — encore sur SQLite
.venv\Scripts\python.exe manage.py reprise_sql exporter --vers reprise

rem 2. basculer .env sur PostgreSQL (section 3), puis créer le schéma
.venv\Scripts\python.exe manage.py migrate

rem 3. charger, avec vérification automatique
.venv\Scripts\python.exe manage.py reprise_sql importer --depuis reprise
```

La sortie de l'étape 1 liste ce qui part, table par table. Celle de l'étape 3 confirme que les comptages sont identiques — ou refuse, sans rien avoir écrit.

> **Répétez la reprise à blanc avant le jour J, sur une copie de db.sqlite3. L'export est en lecture seule : il ne modifie jamais la source.**

Conservez le dossier reprise\ quelques semaines : c'est votre filet si un problème apparaît après la bascule. Contrôle indépendant à tout moment :

```
.venv\Scripts\python.exe manage.py reprise_sql verifier --depuis reprise
```

5. INSTALLER LE SERVICE

Dans une console PowerShell ouverte en administrateur, depuis le dossier du projet :

```
powershell -ExecutionPolicy Bypass -File deploiement\windows\Installer-Serveur.ps1
```

Le script enchaîne : environnement Python, dépendances, fichier .env avec une clé secrète générée et le mode production, bibliothèques du scanner, migrations, contrôle de pré-vol, tâche planifiée et règle de pare-feu. Il termine en interrogeant la sonde de santé et affiche l'adresse à donner aux postes.

Pourquoi une tâche planifiée plutôt qu'un service Windows : elle est native, aucun outil tiers à télécharger ; elle démarre sans session ouverte, donc le WMS repart seul après une coupure de courant ; et elle sait se relancer après un échec.

Commandes utiles ensuite :

```
Get-ScheduledTask -TaskName "WMS Structa" | Get-ScheduledTaskInfo
Start-ScheduledTask -TaskName "WMS Structa"
Stop-ScheduledTask  -TaskName "WMS Structa"
```

6. VÉRIFIER AVANT D'OUVRIR AUX OPÉRATEURS

Le contrôle de pré-vol passe en revue ce qui, en pratique, fait rater une mise en production : clé secrète restée à l'exemple, DEBUG actif, base injoignable, schéma en retard sur le code, aucun compte administrateur, bibliothèques du scanner absentes, journaux non inscriptibles.

```
.venv\Scripts\python.exe outils\preparer_serveur.py
```

Il sort en erreur s'il reste un point bloquant, et Demarrer-serveur.bat refuse alors de lancer le service. Puis, depuis un autre poste du réseau :

> **Le vrai test vient après : connectez-vous, scannez un emplacement, rangez une cabine et vérifiez que le journal l'a enregistré. Une base qui répond ne prouve pas que le métier fonctionne.**

Adresse	Ce que l'on doit obtenir
http://<ip-serveur>:8000/	l'écran de connexion
http://<ip-serveur>:8000/api/sante/	service ok, base ok, migrations ok
http://<ip-serveur>:8000/admin/	l'administration Django

7. SAUVEGARDER

Une sauvegarde qu'on ne sait pas restaurer n'est pas une sauvegarde. Le script vérifie que le fichier produit n'est pas vide et affiche la commande de restauration à chaque passage.

```
powershell -ExecutionPolicy Bypass -File deploiement\windows\Sauvegarder.ps1
-Destination "D:\Sauvegardes\WMS"
```

À planifier une fois par nuit, en tâche SYSTEM :

```
$a = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-
ExecutionPolicy Bypass -File C:\WMS\wms-structa\deploiement\windows\
Sauvegarder.ps1"
$d = New-ScheduledTaskTrigger -Daily -At 22:30
Register-ScheduledTask -TaskName "WMS Structa - sauvegarde" -Action $a -
Trigger $d -User "SYSTEM" -RunLevel Highest
```

Testez une restauration une fois, sur une base d'essai, avant d'en avoir besoin :

```
createdb -U postgres wms_test
pg_restore -U postgres -d wms_test --clean --if-exists D:\Sauvegardes\WMS\
wms_...dump
```

> **Sortez au moins une copie de la machine — disque externe ou partage réseau. Une sauvegarde stockée sur le disque qui tombe ne sert à rien.**

8. SUPERVISER ET DÉPANNER

La sonde /api/sante/ répond 200 si tout va bien, 503 si la base est injoignable ou si le schéma est en retard sur le code. C'est le point à interroger depuis un outil de supervision : elle n'exige aucun compte et ne divulgue ni version, ni chemin, ni hôte de base.

Le fichier logs\wms.log — rotation automatique, cinq fichiers de 2 Mo — est le premier endroit à regarder quand un opérateur signale un problème de la veille.

Symptôme	Cause la plus fréquente
Le service ne démarre pas	Lire logs\wms.log, puis lancer outils\preparer_serveur.py. Neuf fois sur dix : .env incomplet ou base injoignable.
« Le service PostgreSQL ne répond pas »	Le service est arrêté. services.msc, chercher postgresql-x64, démarrer.
« PostgreSQL est introuvable » alors qu'il est installé	« Command Line Tools » n'a pas été coché à l'installation. Les scripts cherchent eux-mêmes dans Program Files\PostgreSQL.
« La base wms existe déjà et contient N tables »	Garde-fou volontaire : le script n'écrase pas une base d'exploitation. Jeter la base à la main, ou relancer avec un autre nom.
Page sans style, boutons inertes	Fichiers statiques non servis. Vérifier DJANGO_SERVIR_STATIQUES=True dans .env.

« duplicate key » après la reprise	Séquences non recalées. reprise_sql importer le fait ; un loaddata lancé à la main, non.
Le scan caméra ne marche pas	La caméra exige HTTPS. En réseau local sur http, la saisie manuelle prend le relais.
« Mes cabines d'hier ont disparu »	Vous n'êtes pas sur la même base. Quelle-Base.bat donne le verdict en trois chiffres.

En cas de doute sur la base active, Quelle-Base.bat affiche le moteur, la base et surtout le nombre d'emplacements, de cabines et d'événements au journal. Les comptages sont le seul repère fiable : deux bases peuvent répondre, une seule contient le travail du jour.